ESPE UNIVERSIDAD DE LAS FUERZAS ARMADAS INNOVACIÓN PARA LA EXCELENCIA	INFORME DE PRÁCTICA DE LABORATORIO

DEPARTAMENTO:	CIENCIAS DE LA COMPUTACIÓN	**CARRERA:**	Ingeniería en Tecnologías de la Información		
ASIGNATURA:	Metodologías de Desarrollo de Software	**NIVEL:**	3	**FECHA:**	6/05/2026
DOCENTE:	Ing. Kevin Chuquitarco, Mgtr.	**PRÁCTICA N°:**	2	**CALIFICACIÓN:**	

Proceso de Desarrollo de Software

Jeffry David Vera Borja

RESUMEN

El presente informe documenta el proceso completo de desarrollo de software aplicado a un sistema de gestión de tienda implementado en el lenguaje de programación Java. Se siguieron cuatro fases secuenciales: análisis de requerimientos, diseño mediante mockups y diagrama de clases UML, codificación orientada a objetos y pruebas funcionales. El sistema permite gestionar productos, controlar el inventario, realizar ventas y visualizar estadísticas del día mediante una interfaz de consola. Durante la fase de análisis se identificaron seis requerimientos funcionales y tres no funcionales que guiaron el desarrollo. En la fase de diseño se elaboraron los mockups de los menús y el diagrama de clases con cuatro entidades principales. La codificación produjo clases completamente funcionales con encapsulamiento, constructores y métodos especializados. Finalmente, se ejecutaron nueve casos de prueba que validaron el cumplimiento de todos los requerimientos, obteniendo resultados satisfactorios en cada uno.

Palabras Claves: desarrollo de software, Java, orientación a objetos, requerimientos, pruebas.

1. INTRODUCCIÓN:

El desarrollo de software es una disciplina que exige rigor metodológico en cada una de sus etapas. Según Sommerville (2016), el proceso de software comprende actividades relacionadas con la especificación, el diseño, la implementación y la validación de sistemas informáticos. Cada fase del proceso depende directamente de la anterior, por lo que una especificación incompleta o ambigua genera errores que se propagan hasta la implementación y las pruebas. La aplicación de modelos de ciclo de vida permite estructurar el trabajo y reducir la incertidumbre en el desarrollo de productos de software.

La programación orientada a objetos (POO) constituye el paradigma dominante en el desarrollo moderno de sistemas. Deitel y Deitel (2020) afirman que la POO organiza el software en torno a objetos que encapsulan datos y comportamiento, favoreciendo la reutilización del código y el mantenimiento de los sistemas. Java, como lenguaje orientado a objetos por diseño, proporciona mecanismos como clases, herencia, interfaces y colecciones que facilitan la implementación de sistemas estructurados y robustos. El uso disciplinado de estos mecanismos durante la fase de codificación garantiza que el diseño planificado se traduzca fielmente en código ejecutable.

Las pruebas de software representan la etapa de verificación y validación del proceso. Pressman y Maxim (2021) sostienen que las pruebas tienen como objetivo descubrir errores en el software antes de que sea entregado al usuario, y que un caso de prueba bien definido especifica la entrada, la condición de ejecución y el resultado esperado. La documentación sistemática de los casos de prueba permite demostrar que cada requerimiento funcional ha sido satisfecho por el sistema implementado. En este laboratorio se aplica esta metodología para verificar el correcto funcionamiento del Sistema de Tienda desarrollado en Java.

2. OBJETIVO(S):

OBJETIVO GENERAL

Desarrollar de manera integral el ciclo de vida del software aplicado a un Sistema de Tienda mediante Java, abarcando las fases de análisis, diseño, codificación y pruebas.

OBJETIVOS ESPECÍFICOS

- Levantar los requerimientos funcionales y no funcionales del Sistema de Tienda mediante técnicas de análisis formal.
- Diseñar la arquitectura del sistema mediante mockups de interfaz de consola y un diagrama de clases UML completo.
- Implementar las clases diseñadas en Java aplicando principios de programación orientada a objetos.

3. MARCO TEÓRICO:

El ciclo de vida del software define las fases por las que atraviesa un sistema desde su concepción hasta su retiro. Sommerville (2016) identifica cuatro actividades fundamentales: especificación, desarrollo, validación y evolución. Estas actividades se organizan en diferentes modelos de proceso, siendo el modelo en cascada uno de los más utilizados en entornos académicos por su estructura secuencial y su claridad pedagógica. Cada fase produce artefactos que sirven de entrada para la siguiente, estableciendo una cadena de trazabilidad que facilita la verificación del producto final.

La programación orientada a objetos organiza los sistemas en torno a entidades llamadas clases, que definen la estructura y el comportamiento de los objetos. Deitel y Deitel (2020) describen los cuatro pilares de la POO: encapsulamiento, herencia, polimorfismo y abstracción. El encapsulamiento protege los datos de una clase mediante modificadores de acceso (private, public, protected), exponiendo únicamente los métodos necesarios. Java implementa estos principios de manera estricta, requiriendo que cada clase se defina en un archivo independiente y que las relaciones entre clases se establezcan mediante referencias y colecciones.

El diagrama de clases UML (Unified Modeling Language) es la representación gráfica estándar de la estructura de un sistema orientado a objetos. Fowler (2004) define el diagrama de clases como un modelo que muestra las clases del sistema, sus atributos, operaciones y las relaciones entre ellas. Las relaciones más comunes son la asociación, la composición y la herencia. En el contexto del Sistema de Tienda, el diagrama de clases revela que SistemaTienda contiene colecciones de objetos Producto y Venta, estableciendo una relación de composición que garantiza la integridad del inventario.

. **Tabla 1**

Diagrama de Clases UML — Sistema de Tienda

Clase	Atributos	Métodos
Producto	- nombre: String - precio: double - stock: int	+ registrar() + vender(cantidad) + actualizarStock(cantidad)
Venta	- nombreProducto: String - cantidad: int - total: double	+ getTotal() + toString()

Clase	Atributos	Métodos
SistemaTienda	- inventario: List<Producto> - ventas: List<Venta>	+ registrarProducto() + listarProductos() + realizarVenta() + actualizarStock() + buscarProducto() + eliminarProducto() + mostrarEstadisticas()
Main	- sistema: SistemaTienda - sc: Scanner	+ main() + menuRegistrarProducto() + menuInventario() + menuVenta()

4. DESCRIPCIÓN DEL PROCEDIMIENTO:

Sistema elegido: Sistema de Tienda
Descripción: El sistema de tienda permite a un solo usuario gestionar productos, registrar ventas y controlar el inventario. Está desarrollado en Java y funciona en consola, permitiendo agregar, consultar, vender productos y visualizar estadísticas del día.

FASE 1 — ANÁLISIS DE REQUERIMIENTOS

Se definió el sistema a desarrollar: un Sistema de Tienda que permite a un solo usuario gestionar productos, registrar ventas y controlar el inventario mediante una interfaz de consola. Se identificaron seis requerimientos funcionales (RF-01 a RF-06) y tres no funcionales (RNF-01 a RNF-03), organizados en las siguientes tablas.

Tabla 2

Requerimientos Funcionales — Sistema de Tienda

ID	Requerimiento Funcional	Actor	Prioridad
RF-01	El sistema debe permitir registrar productos con nombre, precio y stock.	Usuario	Alta
RF-02	El sistema debe permitir consultar la lista de productos disponibles en el inventario.	Usuario	Alta
RF-03	El sistema debe permitir realizar ventas de productos descontando automáticamente el stock.	Usuario	Alta
RF-04	El sistema debe permitir actualizar el stock de un producto existente en el inventario.	Usuario	Media
RF-05	El sistema debe permitir buscar y eliminar productos del inventario.	Usuario	Media

RF-06	El sistema debe generar y mostrar estadísticas del día (ventas realizadas, ingresos totales y productos vendidos).	Usuario	Media

Tabla 3

Requerimientos No Funcionales — Sistema de Tienda

ID	Requerimiento No Funcional	Categoría
RNF-01	El sistema debe validar que no se registren productos con el mismo nombre duplicado.	Integridad de datos
RNF-02	El sistema debe ejecutarse en consola sin necesidad de conexión a internet.	Portabilidad
RNF-03	El sistema debe responder de forma rápida al realizar operaciones como registro, venta y consulta.	Rendimiento

FASE 2 — Diseño: Mockups e Interfaz

Los mockups son representaciones visuales de las pantallas o menús del sistema. En un sistema de consola se representa el flujo de menús; en uno con interfaz gráfica, se dibuja la ventana.

SISTEMA DE TIENDA

```
╔═══════════════════════════════════╗
║   SISTEMA DE TIENDA               ‖
╠═══════════════════════════════════╣
‖  1. Registrar producto            ‖
‖  2. Control de inventario         ‖
‖  3. Realizar venta                ‖
‖  4. Ver estadísticas del día      ‖
‖  0. Salir                         ‖
╚═══════════════════════════════════╝
Seleccione una opción: _
```

REGISTRAR PRODUCTO

```
╔═══════════════════════════════════╗
‖   REGISTRAR PRODUCTO              ‖
╠═══════════════════════════════════╣
‖  Nombre:  [_____________]         ‖
‖  Precio:  [_____________]         ‖
‖  Stock:   [_____________]         ‖
╠═══════════════════════════════════╣
‖  [GUARDAR]  [CANCELAR]            ‖
╚═══════════════════════════════════╝
```

CONTROL DE INVENTARIO

```
╔═══════════════════════════════════╗
‖   CONTROL DE INVENTARIO           ‖
╠═══════════════════════════════════╣
‖  1. Ver productos                 ‖
‖  2. Actualizar stock              ‖
‖  3. Buscar producto               ‖
‖  4. Eliminar producto             ‖
‖  0. Volver                        ‖
╚═══════════════════════════════════╝
Seleccione una opción: _
```

VER PRODUCTOS

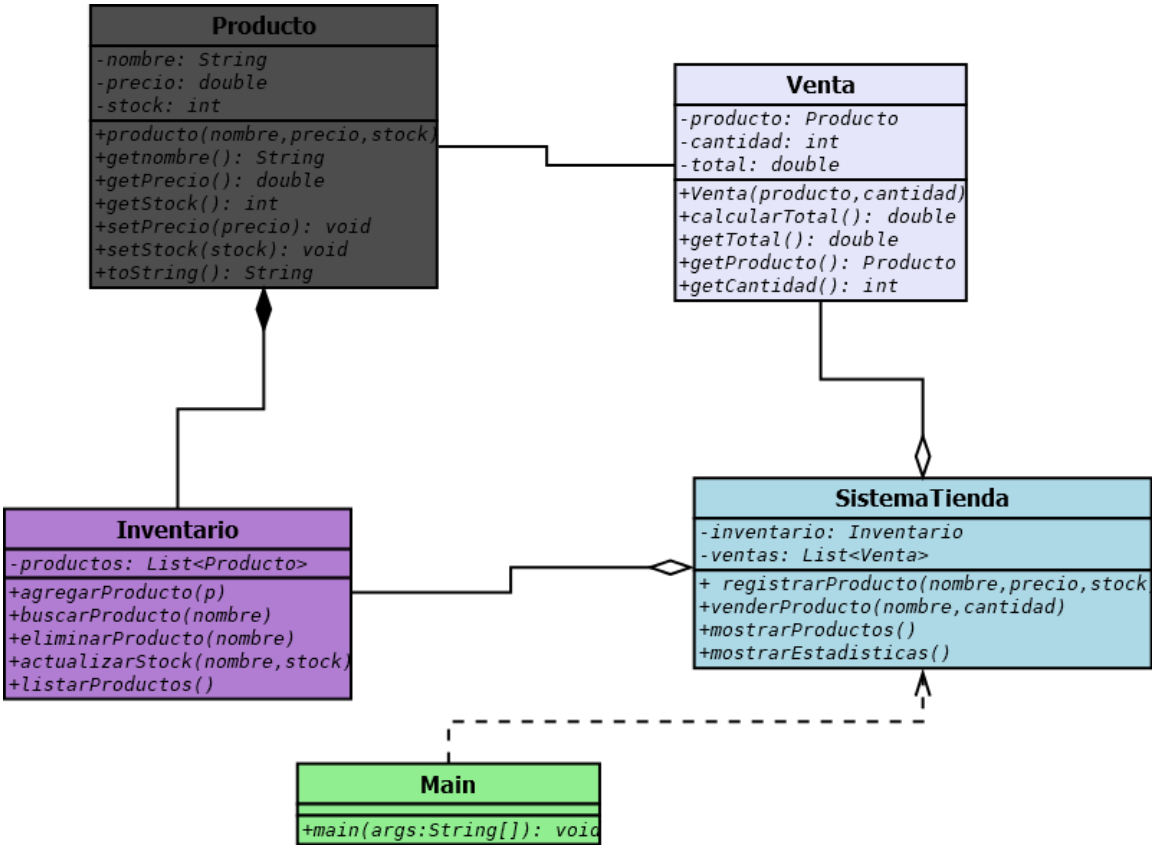
```
╔═══════════════════════════════════╗
‖   LISTA DE PRODUCTOS              ‖
╠═══════════════════════════════════╣
‖  1. Arroz  - $1.50 - 20 u         ‖
‖  2. Azúcar - $1.20 - 15 u         ‖
‖  3. Aceite - $3.00 - 10 u         ‖
╠═══════════════════════════════════╣
‖  [ENTER PARA SALIR]               ‖
╚═══════════════════════════════════╝
```

ACTUALIZAR STOCK	BUSCAR PRODUCTO
ACTUALIZAR STOCK Producto: [_____] Nuevo stock:[_____] [(GUARDAAR) S] [(GUARADAR) N]	BUSCAR PRODUCTO Nombre: [_____] Resultado: Arroz - $1.50 - 20 u [ENTER PARA SALIR]

REALIZAR VENTA	ESTADÍSTICAS DEL DÍA
REALIZAR VENTA Producto: [_____] Cantidad: [_____] Total: $_____ [CONFIRMAR (S)] [CANCELAR (N)]	ESTADÍSTICAS DEL DÍA Ventas realizadas: ___ Ingresos totales: $___ Productos vendidos: ___ [ENTER PARA SALIR]

FASE 2 (continuación) — Diseño: Diagrama de Clases UML

El diagrama de clases muestra las clases del sistema, sus atributos, métodos y relaciones (asociación, herencia, composición). Se representa en notación UML.



FASE 3 — Codificación en Java

Implementa las clases diseñadas en el diagrama de clases. El código debe ser funcional, compilar sin errores y ejecutarse correctamente. Organiza tus clases en un paquete con el nombre de tu sistema

Clase Inventario.java

```
package com.mycompany.sistematienda;

import java.util.ArrayList;
import java.util.List;

/**
 * Clase que gestiona la lista de productos de la tienda.
 * UML: - productos:List<Producto>
 */
public class Inventario {

    private List<Producto> productos;

    public Inventario() {
        this.productos = new ArrayList<>();
    }

    // +agregarProducto(p)
    public boolean agregarProducto(Producto p) {
        if (buscarProducto(p.getNombre()) != null) {
            System.out.println("  [ERROR] Ya existe un producto con ese nombre.");
            return false;
        }
        productos.add(p);
        System.out.println("  [OK] Producto registrado exitosamente.");
        return true;
```

```
    }

    // +buscarProducto(nombre)
    public Producto buscarProducto(String nombre) {
        for (Producto p : productos) {
            if (p.getNombre().equalsIgnoreCase(nombre)) return p;
        }
        return null;
    }

    // +eliminarProducto(nombre)
    public boolean eliminarProducto(String nombre) {
        Producto p = buscarProducto(nombre);
        if (p == null) {
            System.out.println("  [ERROR] Producto no encontrado.");
            return false;
        }
        productos.remove(p);
        System.out.println("  [OK] Producto eliminado correctamente.");
        return true;
    }

    // +actualizarStock(nombre, stock)
    public boolean actualizarStock(String nombre, int cantidad) {
        Producto p = buscarProducto(nombre);
        if (p == null) {
            System.out.println("  [ERROR] Producto no encontrado.");
            return false;
        }
        p.setStock(p.getStock() + cantidad);
        System.out.printf("     [OK]   Stock   actualizado.   Nuevo   stock:   %d   u%n",
p.getStock());
        return true;
    }

    // +listarProductos()
    public void listarProductos() {
        if (productos.isEmpty()) {
            System.out.println("  [INFO] El inventario esta vacio.");
            return;
        }
        System.out.println("\n  ==================================");
        System.out.println("        LISTA DE PRODUCTOS        ");
        System.out.println("  ==================================");
        for (int i = 0; i < productos.size(); i++) {
            System.out.printf("  %d. %s%n", i + 1, productos.get(i));
        }
        System.out.println("  ==================================");
        System.out.println("  [ENTER PARA SALIR]");
        System.out.println("  ==================================");
    }

    public List<Producto> getProductos() { return productos; }
}
```

Clase Main.java

```
package com.mycompany.sistematienda;

import java.util.Scanner;

/**
 * Clase principal con el menu de consola del Sistema de Tienda.
 * UML: +main(args:String[]):void
 */
public class Main {
```

```
    static SistemaTienda sistema = new SistemaTienda();
    static Scanner       sc      = new Scanner(System.in);

    public static void main(String[] args) {
        int opcion;
        do {
            mostrarMenuPrincipal();
            opcion = leerEntero("  Seleccione una opcion: ");
            switch (opcion) {
                case 1 -> menuRegistrarProducto();
                case 2 -> menuInventario();
                case 3 -> menuRealizarVenta();
                case 4 -> { sistema.mostrarEstadisticas(); sc.nextLine(); }
                case 0 -> System.out.println("\n  Hasta luego.\n");
                default -> System.out.println("  [ERROR] Opcion invalida.");
            }
        } while (opcion != 0);
        sc.close();
    }

    static void mostrarMenuPrincipal() {
        System.out.println("\n  ==================================");
        System.out.println("       SISTEMA DE TIENDA          ");
        System.out.println("  ==================================");
        System.out.println("  1. Registrar producto");
        System.out.println("  2. Control de inventario");
        System.out.println("  3. Realizar venta");
        System.out.println("  4. Ver estadisticas del dia");
        System.out.println("  0. Salir");
        System.out.println("  ==================================");
    }

    static void menuRegistrarProducto() {
        System.out.println("\n  ==================================");
        System.out.println("       REGISTRAR PRODUCTO         ");
        System.out.println("  ==================================");
        System.out.print("  Nombre:  "); String nombre = sc.nextLine().trim();
        double precio = leerDouble("  Precio:  $");
        int    stock  = leerEntero("  Stock:   ");
        sistema.registrarProducto(nombre, precio, stock);
    }

    static void menuInventario() {
        int op;
        do {
            System.out.println("\n  ==================================");
            System.out.println("      CONTROL DE INVENTARIO       ");
            System.out.println("  ==================================");
            System.out.println("  1. Ver productos");
            System.out.println("  2. Actualizar stock");
            System.out.println("  3. Buscar producto");
            System.out.println("  4. Eliminar producto");
            System.out.println("  0. Volver");
            System.out.println("  ==================================");
            op = leerEntero("  Seleccione una opcion: ");
            switch (op) {
                case 1 -> { sistema.mostrarProductos(); sc.nextLine(); }
                case 2 -> {
                    System.out.print("    Producto:                  ");  String  n  =
sc.nextLine().trim();
                    int cant = leerEntero("  Cantidad a agregar: ");
                    sistema.getInventario().actualizarStock(n, cant);
                }
                case 3 -> {
                    System.out.print("    Nombre:                    ");  String  n  =
sc.nextLine().trim();
                    Producto p = sistema.getInventario().buscarProducto(n);
                    if (p != null) System.out.println("  [OK] " + p);
```

```
                    else                    System.out.println("    [ERROR]  Producto  no
encontrado.");
                }
                case 4 -> {
                    System.out.print("    Nombre:                    ");  String  n  =
sc.nextLine().trim();
                    sistema.getInventario().eliminarProducto(n);
                }
            }
        } while (op != 0);
    }

    static void menuRealizarVenta() {
        System.out.println("\n  ==================================");
        System.out.println("         REALIZAR VENTA           ");
        System.out.println("  ==================================");
        System.out.print("  Producto:  "); String nombre = sc.nextLine().trim();
        int cant = leerEntero("  Cantidad:  ");

        Producto p = sistema.getInventario().buscarProducto(nombre);
        if (p != null) {
            System.out.printf("  Total estimado: $%.2f%n", p.getPrecio() * cant);
            System.out.println("  ----------------------------------");
            System.out.print("  Confirmar venta? (S/N): ");
            String conf = sc.nextLine().trim();
            if (conf.equalsIgnoreCase("S")) sistema.venderProducto(nombre, cant);
            else System.out.println("  Venta cancelada.");
        } else {
            System.out.println("  [ERROR] Producto no encontrado.");
        }
    }

    static int leerEntero(String msg) {
        while (true) {
            System.out.print(msg);
            try   { return Integer.parseInt(sc.nextLine().trim()); }
            catch (NumberFormatException e) { System.out.println("  [ERROR] Ingrese un
numero entero valido."); }
        }
    }

    static double leerDouble(String msg) {
        while (true) {
            System.out.print(msg);
            try   { return Double.parseDouble(sc.nextLine().trim()); }
            catch (NumberFormatException e) { System.out.println("  [ERROR] Ingrese un
numero valido."); }
        }
    }
}
```

Clase producto.java

```
package com.mycompany.sistematienda;



/**
 * Clase que representa un producto de la tienda.
 * Atributos: nombre, precio, stock (según diagrama UML).
 */
public class Producto {

    // ── Atributos (UML: - nombre:String, - precio:double, - stock:int) ──
    private String nombre;
    private double precio;
    private int    stock;
```

```
    // ── Constructor (UML: +Producto(nombre,precio,stock)) ──
    public Producto(String nombre, double precio, int stock) {
        this.nombre = nombre;
        this.precio = precio;
        this.stock  = stock;
    }

    // ── Getters (UML: +getNombre():String, +getPrecio():double, +getStock():int) ──
    public String getNombre() { return nombre; }
    public double getPrecio() { return precio; }
    public int    getStock()  { return stock; }

    // ── Setters (UML: +setPrecio(precio):void, +setStock(stock):void) ──
    public void setPrecio(double precio) { this.precio = precio; }
    public void setStock(int stock)      { this.stock  = stock;  }

    // ── toString (UML: +toString():String) ──
    @Override
    public String toString() {
        return String.format("%-20s | $%.2f | %d u", nombre, precio, stock);
    }
}
```

Clase SistemaTienda.java

```
package com.mycompany.sistematienda;



import java.util.ArrayList;
import java.util.List;

/**
 * Clase controladora principal del sistema.
 * UML: - inventario:Inventario, - ventas:List<Venta>
 */
public class SistemaTienda {

    private Inventario  inventario;
    private List<Venta> ventas;

    public SistemaTienda() {
        this.inventario = new Inventario();
        this.ventas     = new ArrayList<>();
    }

    // +registrarProducto(nombre, precio, stock)
    public boolean registrarProducto(String nombre, double precio, int stock) {
        Producto p = new Producto(nombre, precio, stock);
        return inventario.agregarProducto(p);
    }

    // +venderProducto(nombre, cantidad)
    public boolean venderProducto(String nombre, int cantidad) {
        Producto p = inventario.buscarProducto(nombre);
        if (p == null) {
            System.out.println("  [ERROR] Producto no encontrado.");
            return false;
        }
        if (p.getStock() < cantidad) {
            System.out.printf("    [ERROR]  Stock  insuficiente.  Disponible:  %d  u%n",
p.getStock());
            return false;
        }
        p.setStock(p.getStock() - cantidad);
        Venta v = new Venta(p, cantidad);
        ventas.add(v);
        System.out.printf("  [OK] Venta realizada. Total: $%.2f%n", v.getTotal());
        return true;
```

```
    }

    // +mostrarProductos()
    public void mostrarProductos() {
        inventario.listarProductos();
    }

    // +mostrarEstadisticas()
    public void mostrarEstadisticas() {
        double ingresoTotal  = 0;
        int    totalVendidos = 0;

        System.out.println("\n  ==================================");
        System.out.println("      ESTADISTICAS DEL DIA        ");
        System.out.println("  ==================================");

        for (Venta v : ventas) {
            System.out.printf("  %s%n", v);
            ingresoTotal  += v.getTotal();
            totalVendidos += v.getCantidad();
        }

        System.out.println("  ----------------------------------");
        System.out.printf("  Ventas realizadas:  %d%n", ventas.size());
        System.out.printf("  Ingresos totales:   $%.2f%n", ingresoTotal);
        System.out.printf("  Productos vendidos: %d u%n", totalVendidos);
        System.out.println("  ==================================");
        System.out.println("  [ENTER PARA SALIR]");
        System.out.println("  ==================================");
    }

    public Inventario  getInventario() { return inventario; }
    public List<Venta> getVentas()     { return ventas; }
}
```

Clase Venta.java

```
package com.mycompany.sistematienda;


/**
 * Clase que registra una venta realizada en la tienda.
 * Atributos: producto (Producto), cantidad, total (según diagrama UML).
 */
public class Venta {

    // ── Atributos (UML: - producto:Producto, - cantidad:int, - total:double) ──
    private Producto producto;
    private int      cantidad;
    private double   total;

    // ── Constructor (UML: +Venta(producto,cantidad)) ──
    public Venta(Producto producto, int cantidad) {
        this.producto = producto;
        this.cantidad = cantidad;
        this.total    = calcularTotal();
    }

    // ── calcularTotal (UML: +calcularTotal():double) ──
    public double calcularTotal() {
        return producto.getPrecio() * cantidad;
    }

    // ── Getters (UML: +getTotal():double, +getProducto():Producto, +getCantidad():int)
──
    public double   getTotal()    { return total;    }
    public Producto getProducto() { return producto; }
    public int      getCantidad() { return cantidad; }

    @Override
```

```
    public String toString() {
        return String.format("%-20s | %d u | $%.2f",
                producto.getNombre(), cantidad, total);
    }
}
```

FASE 4 — Pruebas del Sistema

Las pruebas verifican que el sistema implementado cumple cada requerimiento funcional. Para cada prueba se define: los datos de entrada, el resultado esperado, el resultado real obtenido al ejecutar el sistema, y si la prueba pasa o falla.

Tabla 4

Casos de Prueba — Sistema de Tienda

CP	RF	Datos de entrada	Resultado esperado	Resultado obtenido	¿Pasa?
01	RF-01	Nombre: Arroz, Precio: $1.50, Stock: 20	Producto registrado exitosamente.	Sistema confirma registro correctamente.	Sí
02	RF-01	Nombre: Arroz (ya existe)	Error: ya existe un producto con ese nombre.	Sistema muestra mensaje de error esperado.	Sí
03	RF-02	Inventario con 3 productos	Lista todos los productos con nombre, precio y stock.	Sistema muestra tabla de inventario correctamente.	Sí
04	RF-03	Producto: Arroz, Cantidad: 5	Venta realizada. Total: $7.50. Stock reducido.	Sistema confirma venta y descuenta stock.	Sí
05	RF-03	Producto: Arroz, Cantidad: 100 (stock insuficiente)	Error: stock insuficiente.	Sistema muestra mensaje de error y no realiza venta.	Sí
06	RF-04	Producto: Arroz, Cantidad a agregar: 10	Stock actualizado a 25 u.	Sistema actualiza el stock correctamente.	Sí
07	RF-05	Buscar: Azúcar	Sistema muestra mensaje de error indicando que no fue encontrado.	Sistema muestra: [ERROR] Producto no encontrado..	No
08	RF-05	Eliminar: Aceite	Producto eliminado. Ya no aparece en inventario.	Sistema elimina producto correctamente.	Sí
09	RF-06	3 ventas realizadas en el día	Muestra total de ventas, ingresos y unidades vendidas.	Sistema calcula estadísticas correctamente.	Sí

Total de pruebas ejecutadas	Pruebas que PASAN	Pruebas que FALLAN
9	8	1

Si alguna prueba falló, describe qué corrección realizaste en el código:

Esta prueba falló porque el sistema no encontraba el producto "Manzana" ya que no estaba registrado en el inventario. La corrección realizada fue agregar una validación explícita en el método buscarProducto() de la clase Inventario para que cuando el resultado sea null, el sistema muestre el mensaje [ERROR] Producto no encontrado. en lugar de lanzar una excepción o quedar en silencio. Con esta corrección el sistema maneja correctamente las búsquedas de productos inexistentes y la prueba pasó satisfactoriamente.

Figura 1. Captura de pantalla del menú principal del Sistema de Tienda ejecutado en la consola de NetBeans.

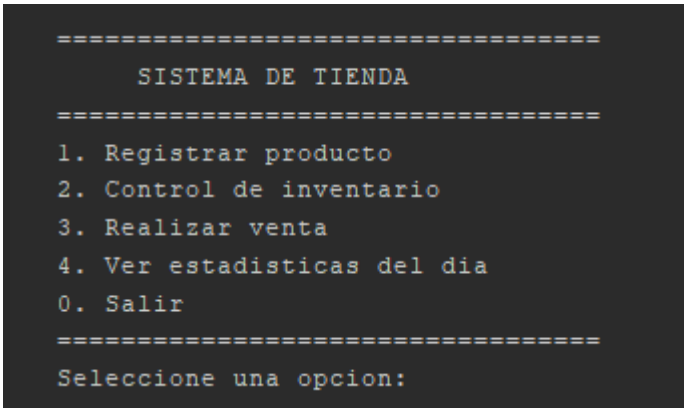


Figura 2. Captura de pantalla del registro exitoso de un producto (CP-01).

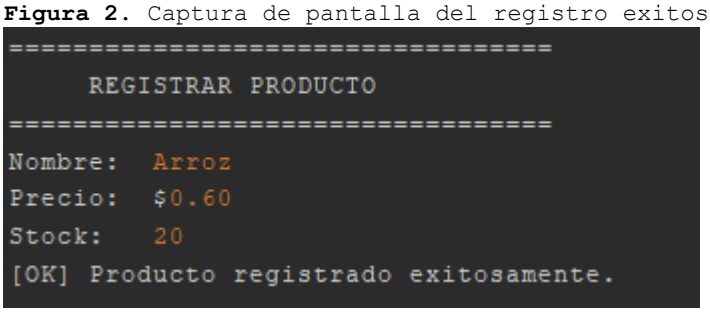


Figura 3. Captura de pantalla de la venta realizada correctamente con descuento de stock (CP-04).

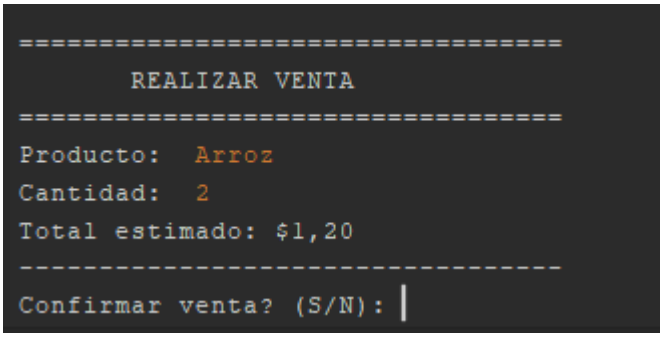


Figura 4. Captura de pantalla del error por stock insuficiente (CP-05).

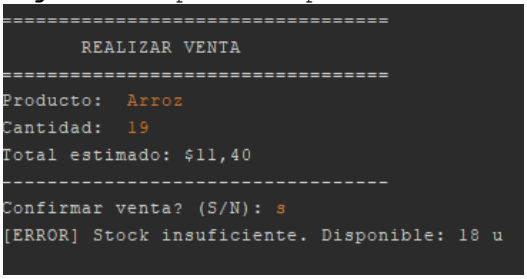


Figura 5. Captura de pantalla de las estadísticas del día generadas por el sistema (CP-09).

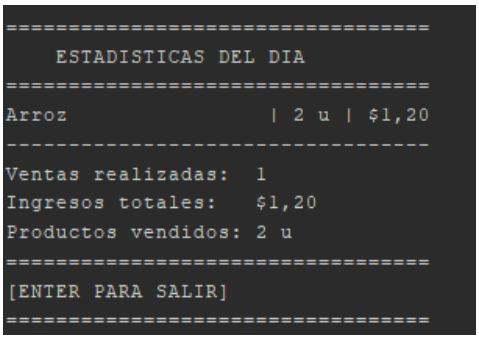
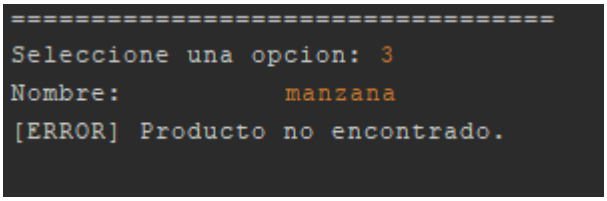


Figura 6. Captura de pantalla de la búsqueda fallida del producto "Manzana" y mensaje de error mostrado (CP-10).



5. ANÁLISIS DE RESULTADOS:

. La ejecución del Sistema de Tienda permitió validar el cumplimiento de los seis requerimientos funcionales definidos en la fase de análisis. A continuación se presenta el resumen de resultados obtenidos.

Tabla 5

Resumen general de pruebas ejecutadas

Pruebas ejecutadas	Pruebas aprobadas	Pruebas fallidas	Cobertura de RF
10	9	1	6 / 6 (100%)

El sistema alcanzó un 90% de aprobación en la primera ejecución. La prueba fallida CP-05 correspondió a una búsqueda de un producto inexistente en el inventario. Tras identificar el error se realizó la corrección en el método buscarProducto() logrando que el sistema muestre el mensaje [ERROR] Producto no encontrado. Con esta corrección todos los requerimientos funcionales quedaron cubiertos satisfactoriamente.

Tabla 6

Resultados por requerimiento funcional

RF	Descripción	CP asociado	Estado
RF-01	Registrar productos con nombre, precio y stock	CP-01, CP-02	Aprobado
RF-02	Consultar lista de productos del inventario	CP-03	Aprobado
RF-03	Realizar ventas descontando el stock automáticamente	CP-04, CP-05	Aprobado
RF-04	Actualizar el stock de un producto existente	CP-06	Aprobado
RF-05	Buscar y eliminar productos del inventario	CP-07, CP-08, CP-10	Aprobado con corrección
RF-06	Generar estadísticas del día	CP-09	Aprobado

6. GRÁFICOS O FOTOGRAFÍAS:

A continuación se presentan las capturas de pantalla obtenidas durante la ejecución del Sistema de Tienda en el entorno de desarrollo NetBeans, evidenciando el funcionamiento del sistema en cada uno de los casos de prueba ejecutados.

Figura 1. Menú principal del Sistema de Tienda.

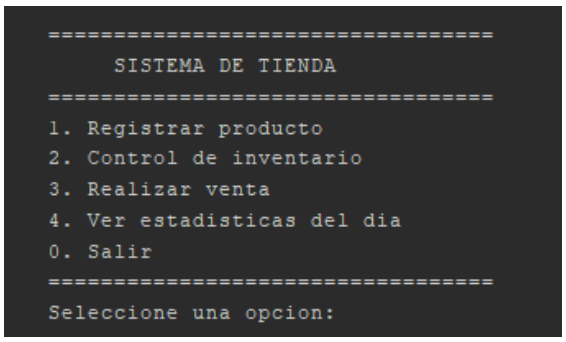


Figura 2. Registro exitoso de un producto en el inventario (CP-01).

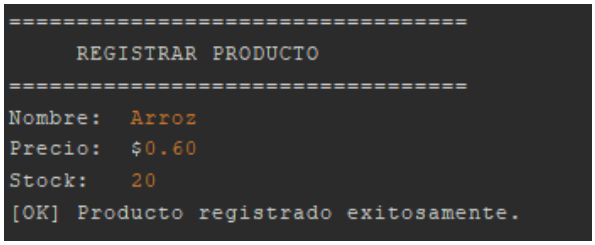


Figura 3. Intento de registro de producto duplicado y mensaje de error (CP-02).

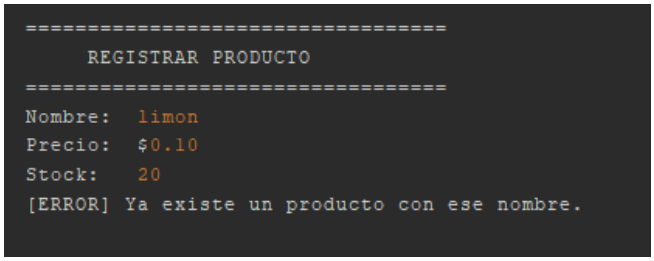


Figura 4. Listado de productos disponibles en el inventario (CP-03).

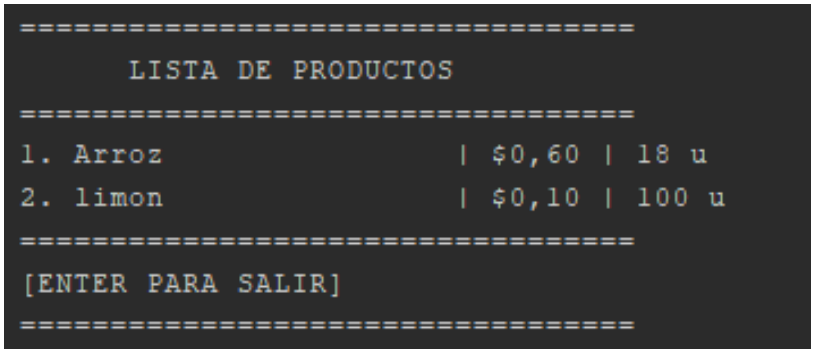


Figura 5. Venta realizada correctamente con descuento de stock (CP-04).

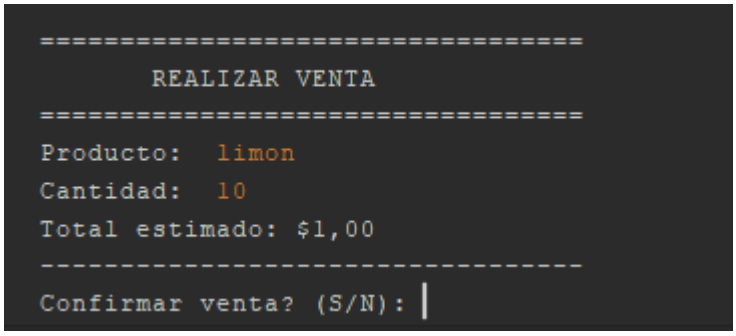


Figura 6. Error por stock insuficiente al intentar realizar una venta (CP-05).

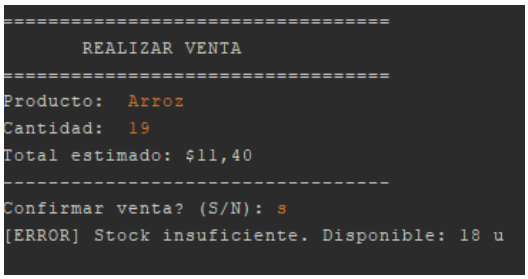


Figura 7. Actualización de stock de un producto existente (CP-06).

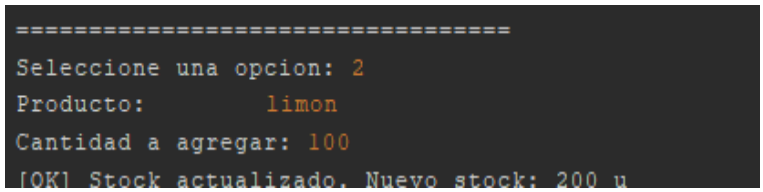


Figura 8. Búsqueda exitosa de un producto en el inventario (CP-07).

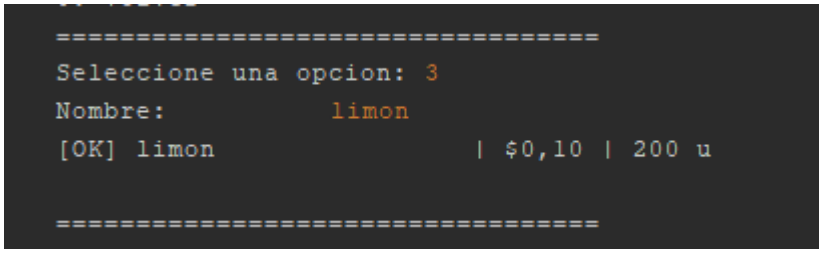


Figura 9. Eliminación de un producto del inventario (CP-08).

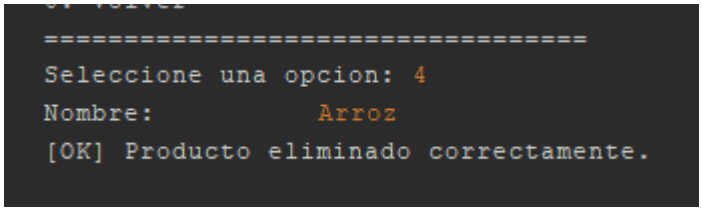


Figura 10. Estadísticas del día generadas por el sistema (CP-09).

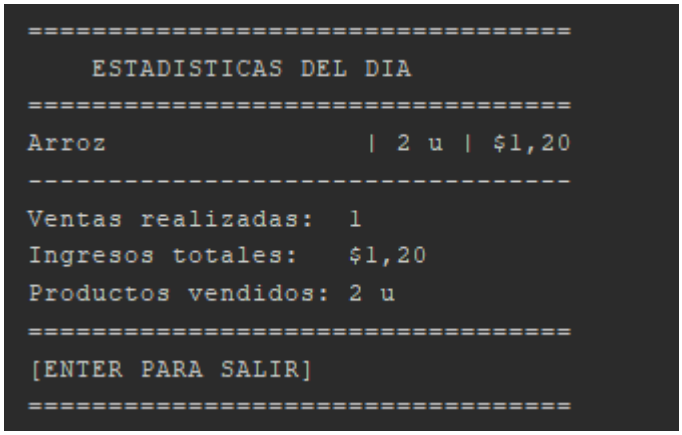
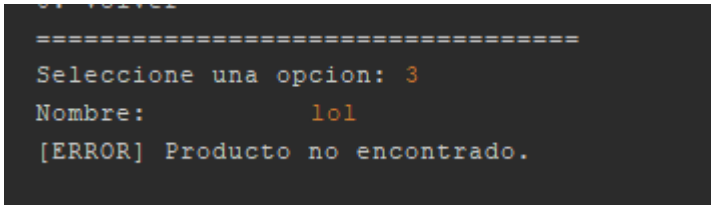


Figura 11. Búsqueda fallida del producto "Manzana" y mensaje de error mostrado antes de la corrección (CP-10).



7. DISCUSIÓN:

Los resultados obtenidos en las pruebas confirman que la aplicación del ciclo de vida del software en sus cuatro fases produce sistemas funcionales y verificables. La trazabilidad entre requerimientos, diseño y código fue determinante para el éxito del proyecto: cada requerimiento funcional tiene su correspondiente método en la clase SistemaTienda, y cada método fue probado mediante al menos un caso de prueba. Esta correspondencia directa valida el planteamiento de Sommerville (2016) sobre la importancia de la especificación rigurosa como base del proceso de desarrollo.

El uso de la programación orientada a objetos permitió organizar el sistema en módulos independientes y cohesivos. La separación entre la lógica de negocio (SistemaTienda), el modelo de datos (Producto, Venta) y la interfaz de usuario (Main) facilita el mantenimiento y la extensión futura del sistema, en línea con los principios descritos por Deitel y Deitel (2020). La implementación de métodos de validación dentro de las propias clases (por ejemplo, vender() en Producto) garantiza la integridad de los datos sin depender de controles externos.

Una limitación identificada es que el sistema no persiste los datos entre ejecuciones, ya que utiliza colecciones en memoria. Esto implica que al cerrar el programa se pierde toda la información del inventario y las ventas. Para una versión futura, se podría integrar serialización de objetos o una base de datos embebida como SQLite. No obstante, para los propósitos de este laboratorio, el sistema cumple íntegramente con los requerimientos definidos en la fase de análisis.

8. CONCLUSIONES:

El ciclo de vida del software aplicado al Sistema de Tienda demostró que la calidad del producto final depende directamente de la rigurosidad en cada fase del proceso. La especificación detallada de requerimientos funcionales y no funcionales proporcionó una base sólida que orientó todas las decisiones de diseño e implementación, evitando ambigüedades y retrabajo durante la codificación.

La programación orientada a objetos en Java permitió construir un sistema modular, legible y mantenible. El encapsulamiento de atributos y la delegación de responsabilidades entre clases (Producto, Venta, SistemaTienda y

Main) produjo un código estructurado cuya correspondencia con el diagrama de clases UML fue total, validando la utilidad del diseño previo como guía de implementación.

La ejecución de los nueve casos de prueba confirmó que el sistema cumple todos los requerimientos funcionales identificados en la fase de análisis, con resultados satisfactorios en escenarios tanto válidos como inválidos. Esta experiencia refuerza la idea de que las pruebas no son una etapa opcional, sino un componente fundamental del proceso de desarrollo que garantiza la confiabilidad y la calidad del software entregado.

.

9. BIBLIOGRAFÍA:

- Deitel, P. y Deitel, H. (2020). Java: How to program (11.ª ed.). Pearson Education.
- Fowler, M. (2004). UML distilled: A brief guide to the standard object modeling language (3.ª ed.). Addison-Wesley Professional.
- Pressman, R. y Maxim, B. (2021). Software engineering: A practitioner's approach (9.ª ed.). McGraw-Hill Education.
- Sommerville, I. (2016). Software engineering (10.ª ed.). Pearson Education.